MISSION: SUMMER BUILD CHALLENGE 2026

Prompt Pack for Building 50 Apps with React Native, Expo Go, and a Build Phase Upgrade Path

Purpose: Use this PDF as your copy-paste prompt guide while building 50 applications during summer break. Each prompt is designed for an AI coding tool such as Antigravity, Cursor, Windsurf, Replit Agent, v0-style builders, or any modern AI IDE. The stack is intentionally aligned with your VakilDesk direction: React Native, Expo Router, Expo Go first, then production build phase later.

Core Development Rules

Rule	Instruction
Expo Go First	Avoid libraries that require prebuild or custom native code unless the prompt explicitly says build phase.
Project Scope	Each app should be useful, clean, and demo-ready, not just a toy UI.
Code Quality	Use reusable components, clear folder structure, TypeScript if possible, and clean error handling.
UI Quality	Use modern cards, tabs, empty states, loading states, and responsive mobile-first layouts.
Showcase Ready	Every app must include README, screenshots folder, features list, and future improvements.
Build Phase	After Expo Go version works, optionally upgrade with native features, EAS Build, notifications, or deeper integrations.

Universal Master Prompt

Paste this before any app-specific prompt. Then paste the specific app prompt from this PDF below it.

```
Act as a senior React Native + Expo developer and product designer.

I am building one app from my project series: summer-build-challenge-2026.
Use the same discipline as a production project, but keep the first version compatible with Expo Go.

Tech Stack Rules:
- React Native with Expo and Expo Router.
- Use TypeScript if the project supports it easily.
- Use functional components and hooks.
- Use clean folder structure: app/, components/, constants/, data/, hooks/, utils/, types/.
- Use AsyncStorage only if local persistence is required and Expo Go compatible.
- Do not use custom native modules in the initial version.
- No expo prebuild, no npx expo run:android, no custom native code in version 1.
- Use clean UI with cards, tabs, empty states, loading states, and error states.
- Include dummy/sample data if backend is not required yet.
- Add comments only where useful.

Output Requirements:
1. Explain the app architecture briefly.
2. Generate the complete code files with paths.
3. Include install commands.
4. Include run commands.
5. Include testing checklist.
6. Include a README.md for this app.
7. Include future build-phase upgrade ideas.

Important: Build the smallest impressive working version first. Do not over-engineer. Make it demo-ready.
```

Folder and README Rule

Inside every app folder, keep a README and screenshots folder. Your main repository can later feed AppDex or your portfolio website from apps.json.

```
Recommended app folder structure:

01-smart-todo-app/
app/
components/
constants/
data/
hooks/
utils/
types/
assets/
screenshots/
README.md
package.json
```

50 App-Specific Prompts

Use each prompt after the Universal Master Prompt. For every app, ask the AI tool to build the Expo Go compatible MVP first. After the MVP runs, ask for build-phase improvements.

Phase 1 - Foundation Apps

App #01 - Smart To-Do App

```
App #01: Smart To-Do App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a simple but polished Smart To-Do App. Include task creation, priority labels, due dates, completed
tasks, daily progress, and local persistence. Use React Native, Expo Router, AsyncStorage, and clean
card-based UI. Add filters for All, Today, Upcoming, Completed. Include empty states and sample tasks.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #02 - Expense Tracker

```
App #02: Expense Tracker

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an Expense Tracker for students. Include income, expenses, category tags, monthly budget, spending
summary, and simple charts or visual progress bars using Expo-compatible libraries. Use local storage first.
Add add/edit/delete transaction screens and a monthly overview.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #03 - Student Notes Organizer

```
App #03: Student Notes Organizer

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Student Notes Organizer. Users can create notes, add subject tags, pin important notes, search notes,
and filter by subject. Store notes locally. The UI should feel like a clean mobile notes app with cards,
search bar, tag chips, and note detail screen.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #04 - Habit Tracker

```
App #04: Habit Tracker

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Habit Tracker with daily check-ins, streak count, weekly progress, and habit categories. Use local
persistence. Include a dashboard, add habit screen, and habit detail screen. Add motivational progress states
and a simple calendar-like weekly view.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #05 - Resume Builder

```
App #05: Resume Builder

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a mobile Resume Builder. Users can enter profile, education, skills, projects, experience, and
achievements. Show a live preview screen. In Expo Go version, provide structured preview and export-ready
content. Build phase can add PDF export.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #06 - Portfolio Generator

```
App #06: Portfolio Generator

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Portfolio Generator app. Users enter name, bio, skills, projects, links, and theme preference.
Generate a clean portfolio preview screen. Allow saving multiple profiles locally. Include project cards and
social link section.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #07 - Password Strength Checker

```
App #07: Password Strength Checker

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Password Strength Checker and Generator. Include strength score, suggestions, password generator
options, copy button, and saved generated passwords locally with warning messages. Keep security educational
and do not transmit passwords anywhere.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #08 - Weather + Outfit Suggestion App

```
App #08: Weather + Outfit Suggestion App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Weather and Outfit Suggestion app. Use a weather API placeholder or mock service first. Show
temperature, condition, humidity, and outfit suggestion based on weather. Include city search, saved cities,
and clean weather cards.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #09 - Movie Recommendation App

```
App #09: Movie Recommendation App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Movie or Anime Recommendation app. Use mock data first or an API-ready service layer. Include search,
genre filters, watchlist, rating, and detail screen. Store watchlist locally. Design the UI like a modern
streaming recommendation app.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #10 - Personal Dashboard

```
App #10: Personal Dashboard

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Personal Dashboard app combining tasks, notes, weather placeholder, quote of the day, and quick links.
Use local data and modular widgets. Include reorder-ready widget design and a clean home screen.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Phase 2 - Full-Stack Ready Apps

App #11 - Assignment Submission Portal

```
App #11: Assignment Submission Portal

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Student Assignment Submission Portal mobile app. Expo Go version should use mock backend services.
Include student login mock, assignment list, upload placeholder, submission status, teacher feedback, and
deadlines. Build phase can connect to Firebase/Supabase.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #12 - College Event Management App

```
App #12: College Event Management App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a College Event Management app. Include event listing, event detail, registration flow, admin mock
screen for creating events, QR-ready registration ID, and favorites. Use local/mock data first with a
backend-ready service layer.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #13 - Lost and Found Campus App

```
App #13: Lost and Found Campus App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Lost and Found Campus app. Users can post found/lost items, add image placeholder, location, category,
contact info, and search/filter items. Use local/mock storage first and make it backend-ready.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #14 - Study Group Finder

```
App #14: Study Group Finder

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Study Group Finder app. Students can browse study groups by subject, join requests, group details,
members, meeting time, and chat placeholder. Use mock data with clean future backend architecture.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #15 - Roommate Finder

```
App #15: Roommate Finder

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Roommate or Flatmate Finder for students. Include profiles, budget, location, habits, preferences,
match score, saved profiles, and contact request. Use mock data and clean filtering UI.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #16 - Freelance Mini Marketplace

```
App #16: Freelance Mini Marketplace

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Freelance Mini Marketplace app. Include task listings, task details, bid submission, user profile,
task status, and client/freelancer views. Use mock data and structure for future backend.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #17 - Appointment Booking App

```
App #17: Appointment Booking App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an Appointment Booking app for teachers, doctors, or lawyers. Include service provider profile,
available slots, booking confirmation, upcoming bookings, and cancellation. Use mock availability data and
backend-ready modules.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #18 - Complaint Management System

```
App #18: Complaint Management System

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Complaint Management System for hostel or college. Include complaint creation, category, priority,
status tracking, admin response, and timeline. Use mock role-based views: student and admin.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #19 - Inventory Management App

```
App #19: Inventory Management App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an Inventory Management app for small shops. Include item list, stock quantity, low-stock alerts,
add/edit item, category filters, and simple sales/stock movement log. Use local/mock storage first.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #20 - Mini CRM

```
App #20: Mini CRM

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Mini CRM for local businesses. Include leads, contacts, follow-up date, lead status, notes, and
dashboard stats. Use local/mock data with clean architecture for Supabase/Firebase later.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Phase 3 - Realtime and Collaboration Apps

App #21 - Realtime Chat App

```
App #21: Realtime Chat App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Realtime Chat app UI first. Include conversations, chat detail, message bubbles, typing indicator
mock, online status, and group chat screen. Keep Expo Go compatible. Build phase can add Firebase/Supabase
realtime.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #22 - Collaborative Notes App

```
App #22: Collaborative Notes App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Collaborative Notes app. Expo Go version should simulate collaboration with local users and version
history. Include shared notes, editor, comments, last edited info, and conflict warning UI. Build phase can
add realtime syncing.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #23 - Live Polling App

```
App #23: Live Polling App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Live Polling app for classrooms and events. Include create poll, vote screen, live results
visualization, poll history, and QR/code join flow placeholder. Use mock realtime state first.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #24 - Kanban Project Manager

```
App #24: Kanban Project Manager

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Kanban Project Manager mobile app. Include boards, columns, task cards, status movement, task detail,
labels, due dates, and team member mock avatars. Build it with local state first and backend-ready services.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #25 - Code Snippet Sharing App

```
App #25: Code Snippet Sharing App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Code Snippet Sharing app. Include snippet list, language tags, syntax-style display, favorites,
search, and snippet detail. Use mock data. Add future plan for auth and cloud sharing.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #26 - Realtime Quiz Battle App

```
App #26: Realtime Quiz Battle App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Realtime Quiz Battle app. Include lobby screen, quiz categories, countdown, question screen, score
screen, leaderboard, and match result. Use simulated realtime behavior locally.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #27 - Attendance Tracker

```
App #27: Attendance Tracker

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a College Attendance Tracker. Include teacher and student views, class list, mark attendance UI,
attendance percentage, subject-wise stats, and low attendance warnings. Use mock data with future realtime
backend plan.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #28 - Team Task Manager

```
App #28: Team Task Manager

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Team Task Manager. Include projects, assigned tasks, status, comments placeholder, notifications
placeholder, and dashboard. Use local/mock data but design the structure for Supabase realtime.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Phase 4 - AI Utility Apps

App #29 - AI Email Writer

```
App #29: AI Email Writer

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Email Writer app. Expo Go version should include input fields for purpose, recipient, tone,
length, and generated email output. Use a mock AI service by default, with a clean function where
OpenAI/Gemini API can be added later. Include copy and save draft.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #30 - AI Resume Reviewer

```
App #30: AI Resume Reviewer

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Resume Reviewer app. Include resume text input, job role input, review categories, score screen,
improvement suggestions, and action checklist. Use mock AI response first and create a service layer for LLM
API integration.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #31 - AI Interview Practice App

```
App #31: AI Interview Practice App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Interview Practice app. Include role selection, question generation mock, answer input, feedback
screen, score, and practice history. Build voice as a future build-phase feature, not in Expo Go version
unless using Expo-compatible APIs.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #32 - AI Study Planner

```
App #32: AI Study Planner

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Study Planner app. User enters subjects, exam date, weak topics, daily availability, and target
score. App generates a study timetable, daily tasks, revision plan, and progress tracker. Use mock AI planner
service first.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #33 - AI Code Explainer

```
App #33: AI Code Explainer

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Code Explainer app. User pastes code, selects language and explanation level, and receives simple explanation, bug risks, and improvement suggestions. Use mock AI response first and keep LLM API integration isolated.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG, notifications, file upload, or deployment depending on the app.
```

App #34 - AI Legal Notice Drafting App

```
App #34: AI Legal Notice Drafting App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Legal Notice Drafting app. User selects notice type, fills facts, parties, dates, and desired tone. App generates a structured legal notice draft. Use safe disclaimers, mock AI generation first, and future LLM integration plan.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG, notifications, file upload, or deployment depending on the app.
```

App #35 - AI Meeting Summary App

```
App #35: AI Meeting Summary App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Meeting Summary app. User pastes transcript or notes. App returns summary, decisions, action
items, owners, and follow-up tasks. Use mock AI output first and include copy/export-ready UI.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #36 - AI WhatsApp Reply Tool

```
App #36: AI WhatsApp Reply Tool

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI WhatsApp Reply Suggestion Tool. User enters received message and desired tone. App generates
short, polite, professional, friendly, or firm replies. Use mock AI output and keep it privacy-first.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Phase 5 - RAG-Based Apps

App #37 - Chat with PDF App

```
App #37: Chat with PDF App

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Chat with PDF mobile app UI and architecture. Expo Go version should support document selection
placeholder, extracted text mock, chat interface, source snippet display, and local sample PDF content. Build
phase can add backend RAG with embeddings and vector DB.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #38 - Chat with College Notes

```
App #38: Chat with College Notes

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Chat with College Notes app. Include subject selection, uploaded notes placeholder, semantic search
mock, Q&A chat, citations/snippets UI, and generated quiz from notes. Keep Expo Go version frontend plus
service abstraction.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #39 - Legal Document Analyzer

```
App #39: Legal Document Analyzer

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Legal Document Analyzer app. Include upload placeholder, document summary, risk points, important
dates, parties involved, and Q&A with source snippets. Add strong disclaimer that it is not legal advice.
Future build phase: OCR + RAG backend.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #40 - Research Paper Summarizer

```
App #40: Research Paper Summarizer

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Research Paper Summarizer and Q&A app. Include paper input/upload placeholder, abstract summary, key
contributions, methodology, limitations, citations/snippets UI, and question chat. Use mock parsing first.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #41 - Personal Knowledge Base

```
App #41: Personal Knowledge Base

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Personal Knowledge Base app. User can add notes, links, documents placeholder, tags, and ask questions
across knowledge. Expo Go version should implement local notes and mock semantic answers. Future build phase:
vector DB.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #42 - Company Policy Chatbot

```
App #42: Company Policy Chatbot

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Company Policy Chatbot app. Include policy categories, document library placeholder, chat Q&A, source
references, role-based access mock, and escalation option. Use mock policy data and backend-ready RAG
structure.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #43 - Syllabus Exam Prep Bot

```
App #43: Syllabus Exam Prep Bot

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build a Syllabus-Based Exam Preparation Bot. Include syllabus input, unit-wise notes, important questions,
quiz generation, short answers, and revision planner. Use mock AI/RAG responses first and design future
document-based RAG.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Phase 6 - Agentic and Advanced Apps

App #44 - AI Personal Assistant

```
App #44: AI Personal Assistant

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Personal Assistant app. Include tasks, reminders placeholder, notes, daily plan generation, and
multi-step assistant responses. Use a mock agent service that shows plan, actions, and final answer. Future
build phase can connect tools and calendar.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #45 - AI Travel Planner

```
App #45: AI Travel Planner

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Travel Planner app. User enters destination, budget, days, interests, and travel style. App
generates itinerary, daily plan, packing list, and estimated budget. Use mock AI planner first and future APIs
for maps/weather.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #46 - AI Finance Coach

```
App #46: AI Finance Coach

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Finance Coach app. Include expense input, monthly budget, spending categories, insights, saving
suggestions, and financial habit score. Use local data and mock AI coach responses. Add clear disclaimer:
educational, not financial advice.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #47 - AI Customer Support Bot

```
App #47: AI Customer Support Bot

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Customer Support Bot app. Include customer chat, ticket creation, FAQ knowledge base mock,
escalation, and admin ticket dashboard. Future build phase: RAG plus CRM integration.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #48 - AI Career Roadmap Builder

```
App #48: AI Career Roadmap Builder

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI Career Roadmap Builder. User enters current skills, target role, time available, and preferred
tech. App generates roadmap, weekly milestones, projects, and skill gap analysis. Use mock AI planner first.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

App #49 - AI App Builder Prompt Generator

```
App #49: AI App Builder Prompt Generator

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build an AI App Builder Prompt Generator. User enters app idea, platform, features, tech stack, and
difficulty. App generates clean prompts for AI coding tools, file structure, MVP plan, and build checklist.
This can help future apps.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Phase 6 - Final Capstone

App #50 - VakilDesk Lite

```
App #50: VakilDesk Lite

Build this app for my summer-build-challenge-2026 series.

Main Goal:
Build VakilDesk Lite as the final capstone. React Native Expo Go first. Include lawyer dashboard, client list,
case list, hearing dates, document upload placeholder, AI summary mock, legal document Q&A mock, reminders,
and WhatsApp/email draft screen. Keep architecture ready for Firebase, OCR, RAG, and Calendar API in build
phase.

Required Screens:
- Home/Dashboard screen
- Add or input screen if relevant
- Detail or result screen if relevant
- Settings/About screen with challenge info

Required Quality:
- Mobile-first polished UI
- Expo Go compatible MVP
- Clean folder structure
- Reusable components
- Dummy/sample data where backend is not yet required
- Clear error, loading, and empty states
- README.md with features, tech stack, run commands, screenshots section, and future improvements

Future Build Phase:
Suggest how this app can be upgraded after Expo Go MVP using backend, native build, AI API, RAG,
notifications, file upload, or deployment depending on the app.
```

Reusable Upgrade Prompts

After an Expo Go version works, use these prompts to improve the app safely.

Debug and Stabilize

```
Review the existing Expo React Native project. Find errors, broken imports, bad file paths, runtime issues, and UI problems. Fix the app without changing the main product idea. Keep it Expo Go compatible unless I explicitly ask for build-phase features.
```

Improve UI/UX

```
Redesign the app UI to look modern, clean, and portfolio-ready. Keep the same features, but improve spacing, typography, colors, cards, navigation, empty states, loading states, and mobile usability. Do not add unnecessary complexity.
```

Add Firebase/Supabase Backend

```
Upgrade this Expo app from mock/local data to a real backend. Add authentication, database models, CRUD operations, loading/error states, and security-minded data separation. Keep the code clean and explain required environment variables.
```

Add AI API Integration

```
Replace the mock AI service with a real LLM API integration through a secure backend route. Do not expose API keys in the mobile app. Add prompt structure, error handling, rate-limit handling, and safe fallback responses.
```

Add RAG Backend

```
Design and implement a backend RAG flow for this app. Include document ingestion, chunking, embeddings, vector storage, retrieval, source snippets, and answer generation. Keep the mobile app as the frontend and explain deployment steps.
```

Prepare for GitHub Showcase

```
Prepare this app for GitHub and portfolio showcase. Add a strong README, screenshots section, feature list, tech stack, learning notes, known limitations, future improvements, and demo instructions.
```

Final Advice

Do not ask the AI tool to build everything at once. Use one app prompt at a time. First get the app running. Then improve UI. Then add persistence/backend. Then prepare README and screenshots. This rhythm will help you actually finish all 50 apps.